Steam Game Dataset

By:
Frankie Snedecor

Motivation

I have always had an interest in gaming and game development. I have a somewhat strong knowledge as what makes a game popular, but some others don't. My goal of this project is to use game statistics from Steam to predict what makes a game popular, and show important features to focus on.

?

!

The Dataset

I will be using a dataset provided from Kaggle, that has a list of over 200k games and specifics about each game.

These features include, price, genre, date released, positive reviews, game name, etc.

'total_positive_reviews', 'metacritic_score', and 'recommendations_total' are my key features for the model as they are critical in classifying success.

Total there are about 73 features over 13 csv files, but majority of them I won't be using as they aren't relevant to my models goal.

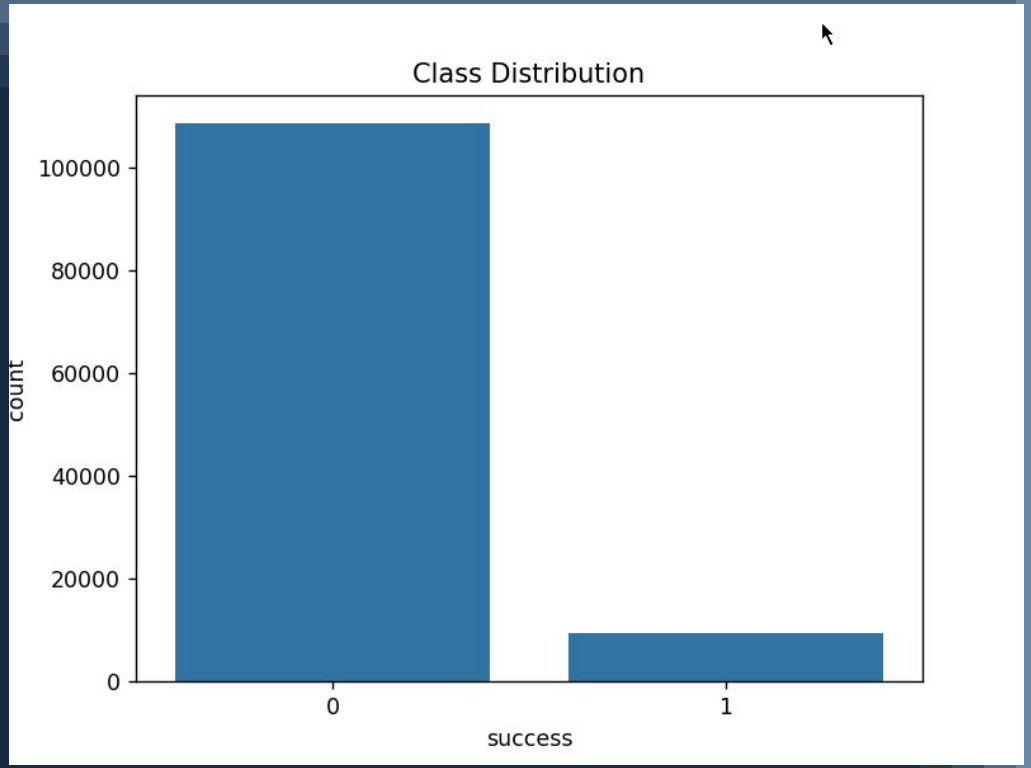
kaggle

Problem Type & Metrics

In my classification report, I will focusing on, precision, recall, f1-score, and accuracy.

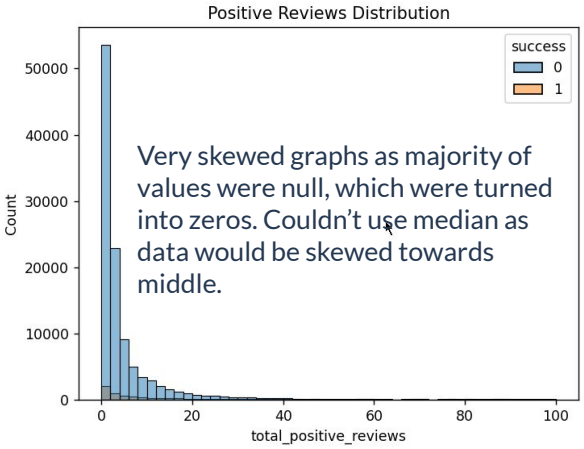
These metrics are important to understand whether the model is correctly classifying the data.

For instance since the data here is very skewed and unbalanced, accuracy won't be a useful statistic but I will still include it as it can show how accurate the models are showing unsuccess.

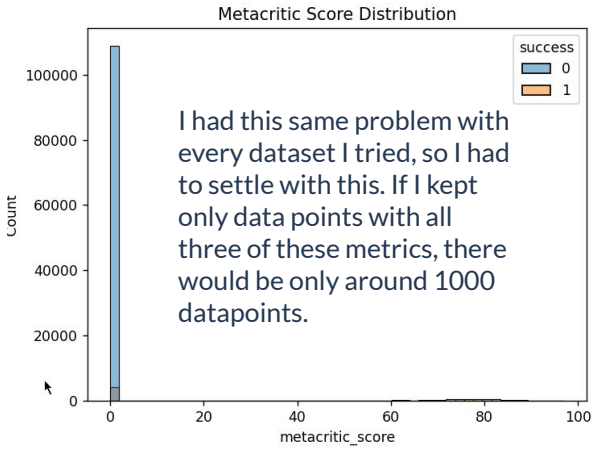


I will be using classification, classifying whether a game is successful (1) or unsuccessful (0).

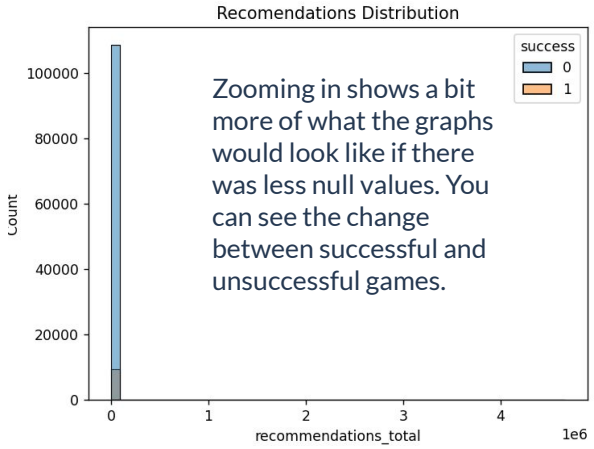
Exploratory Data Analysis



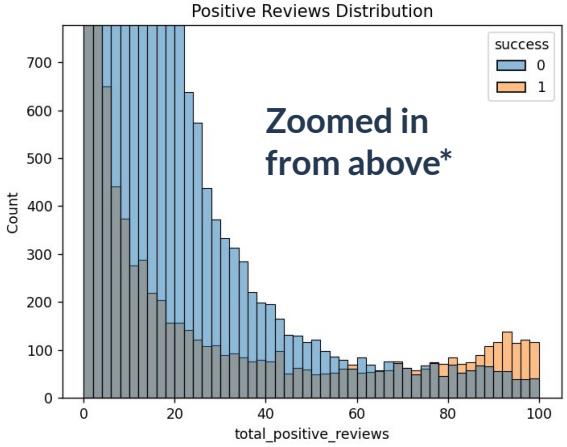
Very skewed graphs as majority of values were null, which were turned into zeros. Couldn't use median as data would be skewed towards middle.



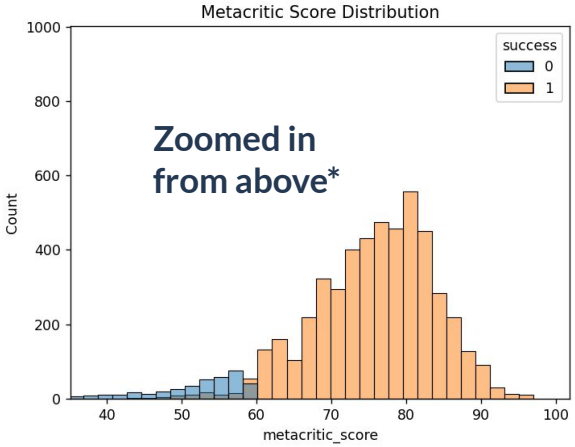
I had this same problem with every dataset I tried, so I had to settle with this. If I kept only data points with all three of these metrics, there would be only around 1000 datapoints.



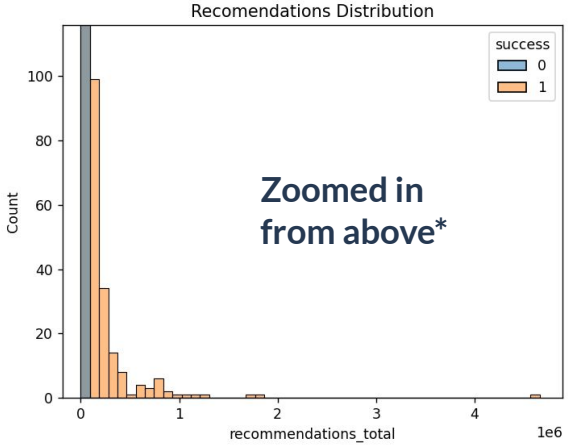
Zooming in shows a bit more of what the graphs would look like if there was less null values. You can see the change between successful and unsuccessful games.



Zoomed in from above*



Zoomed in from above*



Zoomed in from above*

Pre-processing

Train/test split ratio

80% of data goes to training

20% of data goes to testing

Standard splitting process, 75% 25% could be a possibility as well as that's very common, but stuck with 80/20 since I have a larger dataset.

Data Cleaning

Removed duplicates

I made sure each datapoint had at least one of the critical data points:

'total_positive_reviews'
'metacritic_score'
'recommendations_total'

If there was a datapoint missing from all three, it was removed.

Null objects were filled with "No Data" and numerical values were filled with 0

Feature Engineering

Price brackets based on how expensive a game is.
Free, $, $$, $$$, $$$$
↑
$0, $10, $30, $60, $100

If a game is recent based on year released. If date is above 2020 then it's considered "recent"

A ratio of positive and negative reviews for that game.

Validation

For my hyperparameters, I used cross validation.

This is to prevent overfitting and to find which value works best for the model.

Each model will be using around 100k data points, so I made my parameter values larger for each parameter. (Shown in next slide)

Decision Tree

```
grid_dt = GridSearchCV(dt, param_grid_dt, cv=5, scoring='f1')
grid_dt.fit(X_train, y_train)
print("Chosen parameters:", grid_dt.best_params_)
```

Random Forest

```
grid_rf = GridSearchCV(rf, param_grid_rf, cv=5, scoring='f1')
grid_rf.fit(X_train, y_train)
print("Chosen parameters:", grid_rf.best_params_)
```

Model 1, Decision Tree

For my first model I used decision tree. I choose this since it's a great model for classification.

max_depth is how deep the tree goes

min_samples_split is how many data points for each node in the tree

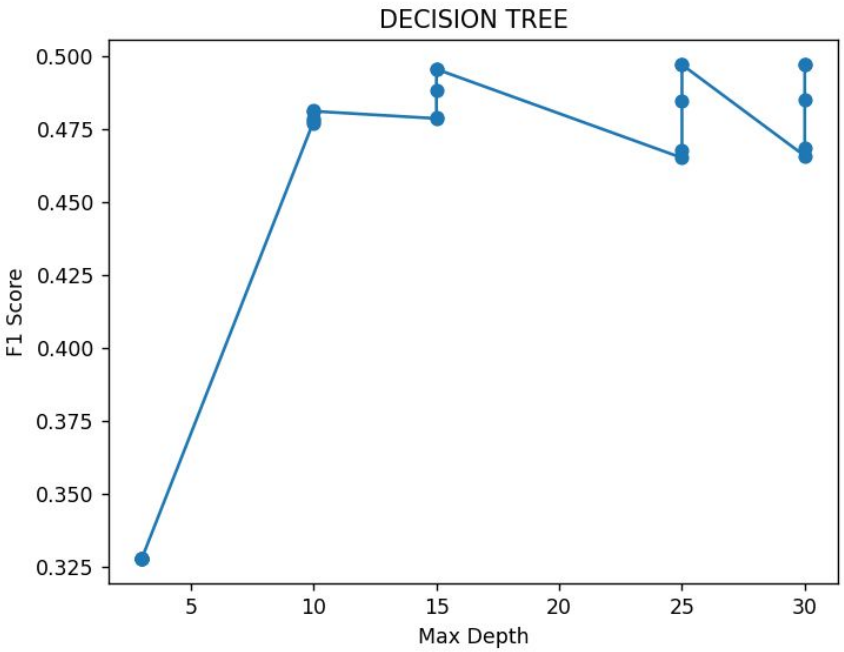
```
param_grid_dt = {
    'max_depth': [3, 10, 15, 25, 30],
    'min_samples_split': [2, 5, 10, 50, 100]
}
```

?

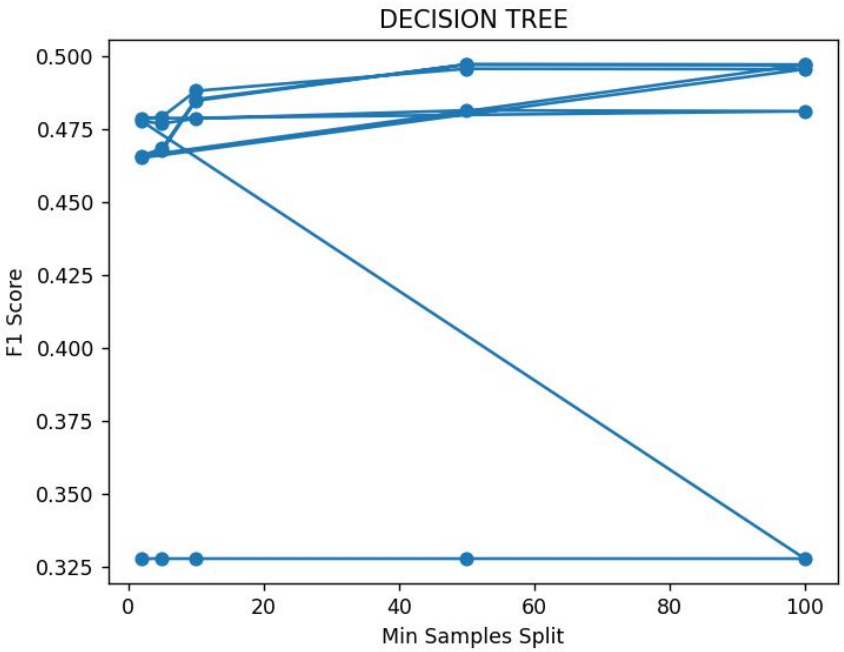
!

Model 1 Hypertuning Results

Tuning max_depth

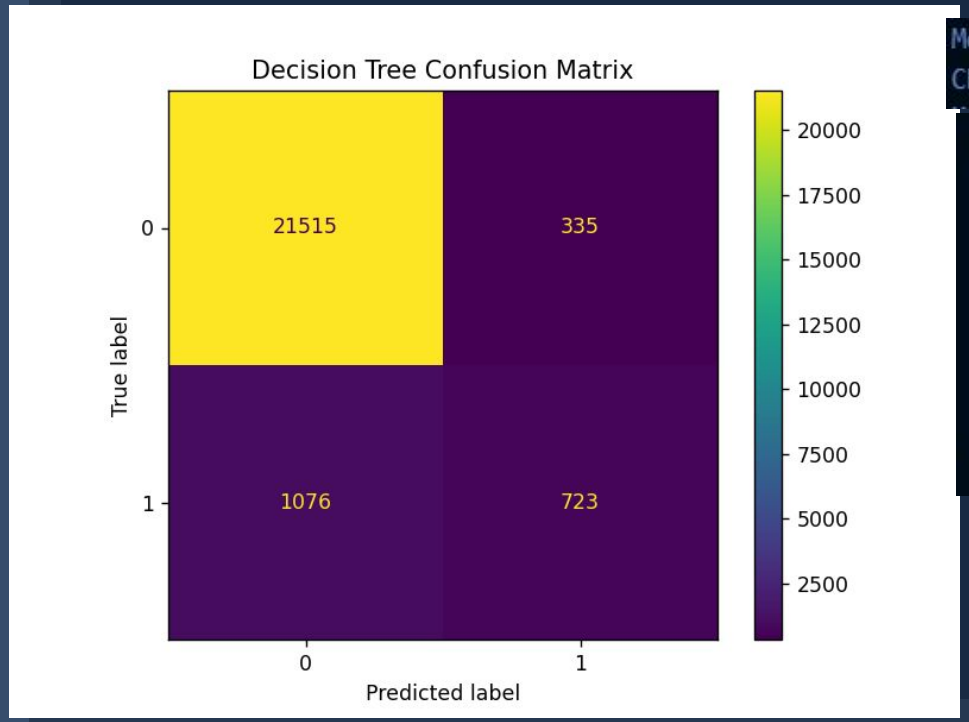


Tuning min_samples_split



Model 1 Results

Decision Tree Matrix



Classification Report

```
Model 1: Decision Tree
Chosen parameters: {'max_depth': 25, 'min_samples_split': 50}
```

Stats from Decision Tree

	precision	recall	f1-score	support
0	0.95	0.98	0.97	21850
1	0.68	0.40	0.51	1799
accuracy			0.94	23649
macro avg	0.82	0.69	0.74	23649
weighted avg	0.93	0.94	0.93	23649

Model 2, Random Forest

For my second model I used random forest. I wanted to see the difference between the two models, a single tree vs an ensemble of trees. I would assume random forest would have better results as it's more randomized.

Max depth is how deep the forest can go

n_estimators is how many trees are created.

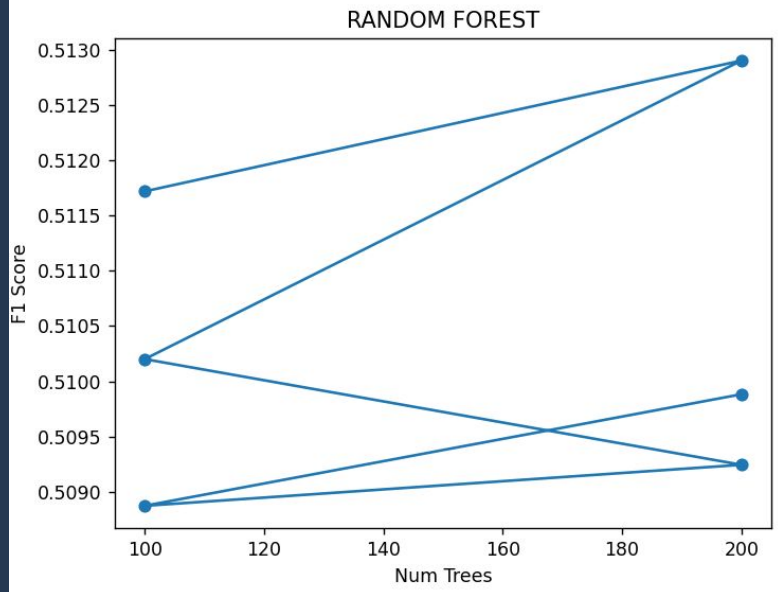
```
param_grid_rf = {
    'max_depth':[15, 25, 30], #
    'n_estimators': [100, 200]
}
```

?

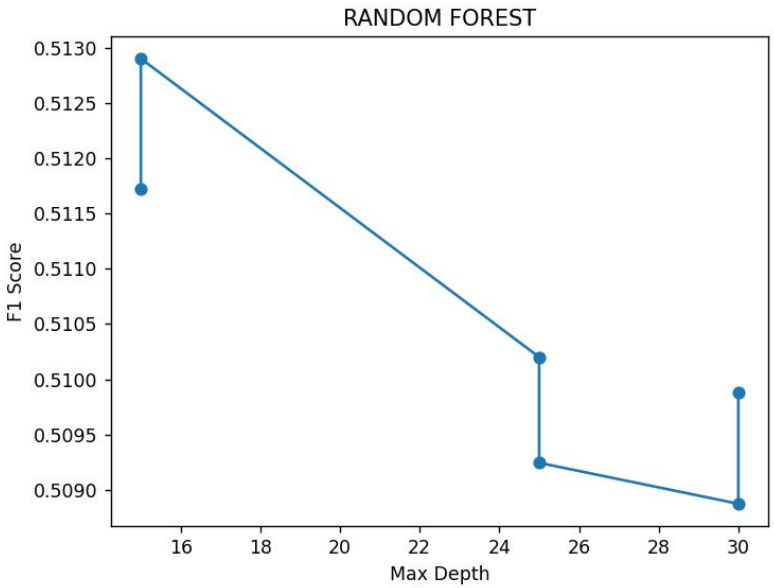
!

Model 2 Hypertuning

Tuning n_estimators (num trees)

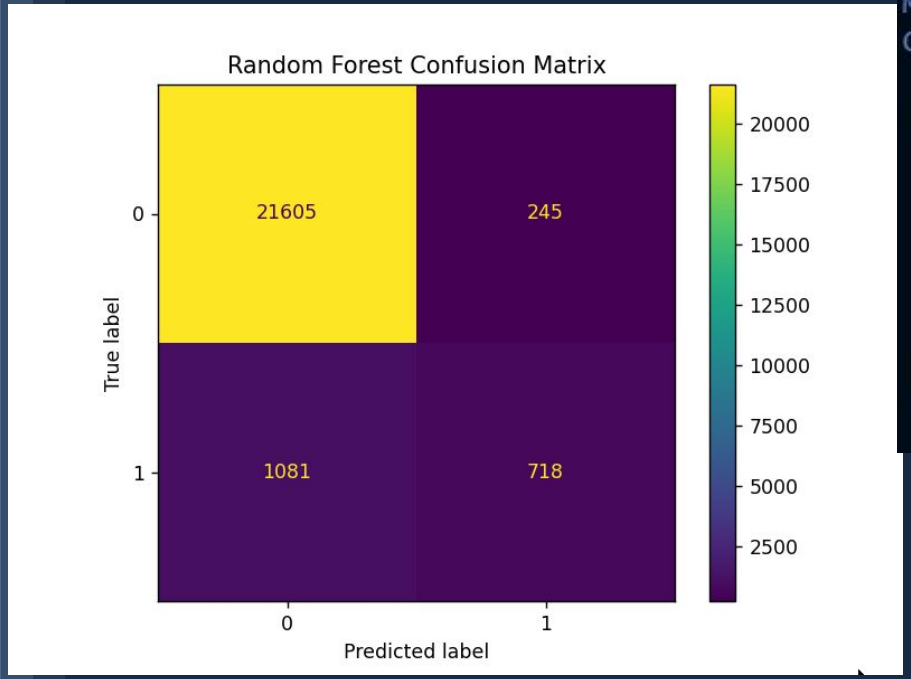


Tuning max_depth



Model 2 Results

Random Forest Matrix



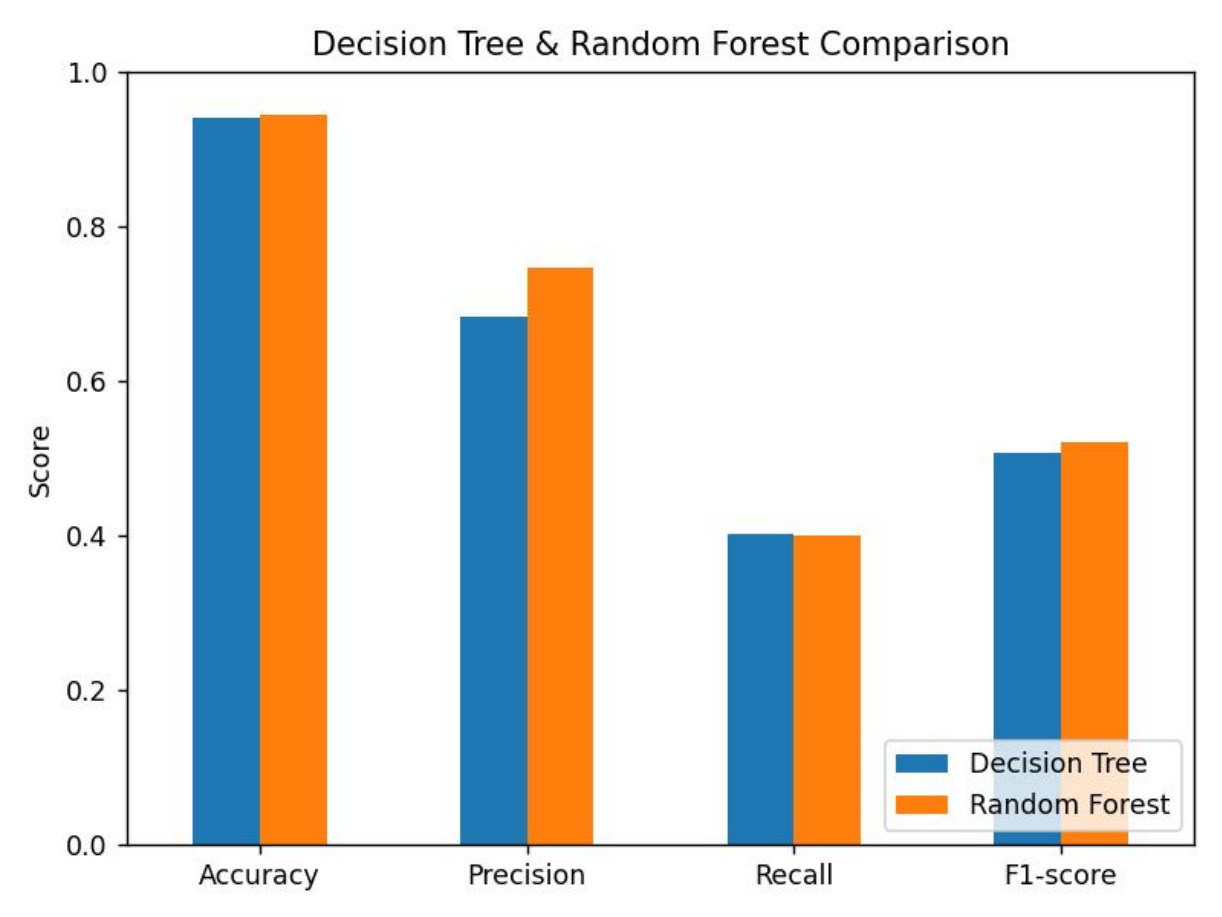
Classification Report

```
Model 2: Random Forest
Chosen parameters: {'max_depth': 15, 'n_estimators': 200}
 Stats from Random Forest
              precision    recall  f1-score   support

           0       0.95      0.99      0.97     21850
           1       0.75      0.40      0.52      1799

    accuracy                           0.94     23649
   macro avg       0.85      0.69      0.75     23649
weighted avg       0.94      0.94      0.94     23649
```

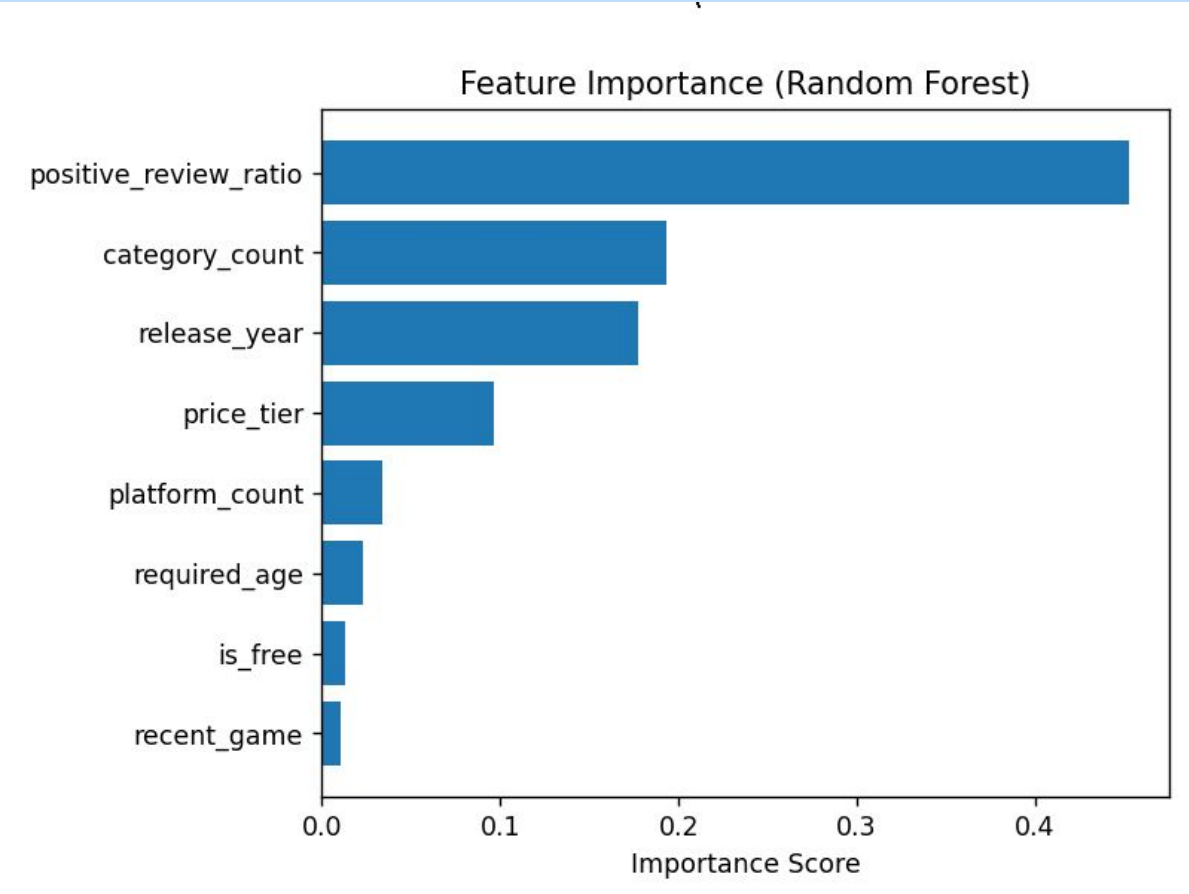
Model Comparison



Originally I predicted that random forest would be the better model and here it shows its slightly performing better across the board.

The only drawback I ran into was random forest took a lot longer, almost 5 minutes longer, than decision tree. This is because my hyperparameters were very large and had to be tuned with each one.

Conclusion and Takeaways



Decision Tree Feature Importance:

	Feature	Importance
7	positive_review_ratio	0.401584
2	release_year	0.228074
5	category_count	0.141990
0	price_tier	0.139907
6	platform_count	0.031886
4	required_age	0.031502
1	is_free	0.022589
3	recent_game	0.002467

By far positive review ratio is the most important statistic a game needs to be successful. Followed by how many categories it has, for example, multiplayer, cross platform, etc, or the release year.

Problems & Future Work

Switched between multiple data sets (lack of data in every dataset)

Very unbalanced models, as shown in classification model plot.

Learned that balanced data makes a better model, and graphs look nicer.

Spent more time trying to find a data set that works rather than enhancing my models and features.

<u>Future work:</u>

Some ideas weren't met

Implement the ability to compare a pre selected game and if it would be successful.

Being able to see top genres, top pricing, etc.

?

!

Thank you

Citations:

Kaggle Dataset: https://www.kaggle.com/datasets/abhishekgupta56447/steam-games-dataset-2025/data

Original Data Source: Steam Web API.

This dataset(Kaggle) is a CSV export of the research record found at: zenodo.org/records/17266923.

https://www.geeksforgeeks.org/

Github:
https://github.com/frankiesned/Steam-Game-Popularity-Prediction-Model